

SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert

LECTURE 2: TEXT MINING

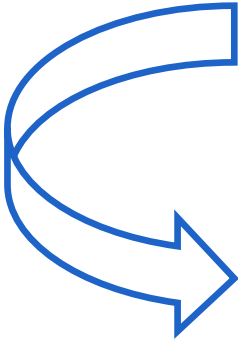
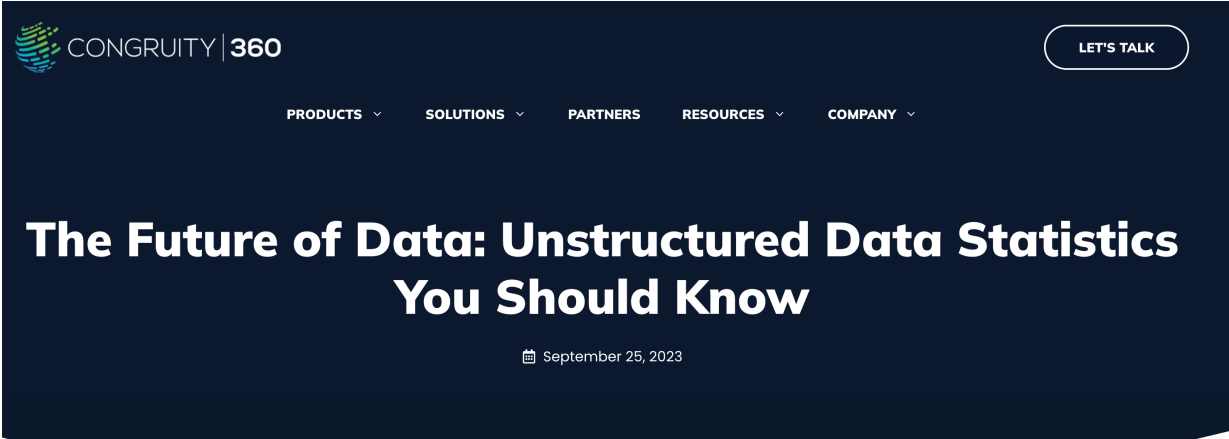
CONTENT

- Lecture 2: Text mining
 - Text mining
 - Introduction
 - Bag-of-words and TF-IDF
 - SVD
 - Word Analysis
 - Word clouds
 - Word networks
 - Re-tweet analysis

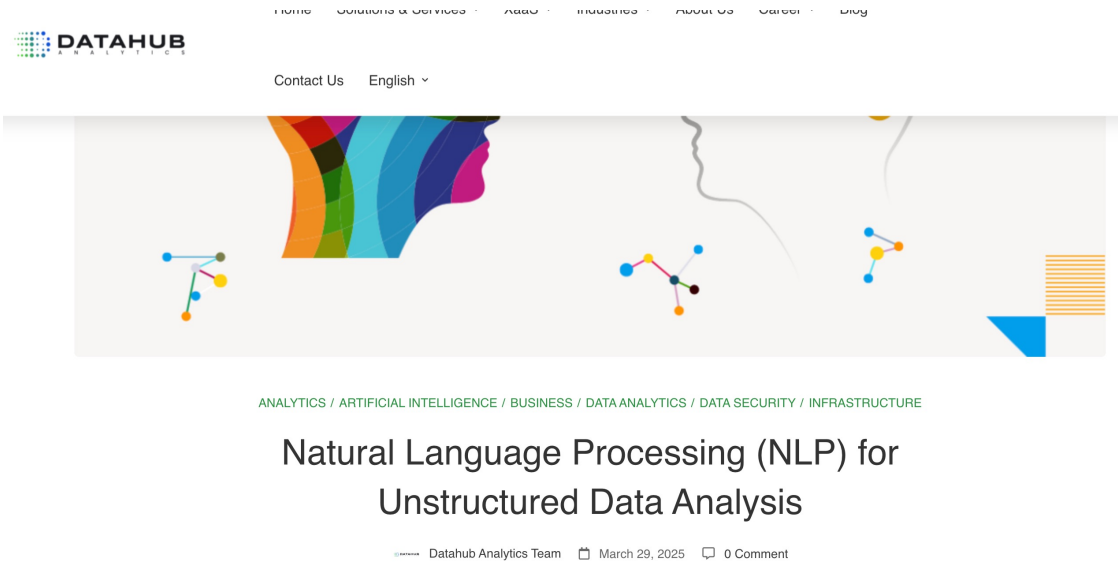
TEXT MINING: INTRODUCTION

WHY TEXT MINING MATTERS?

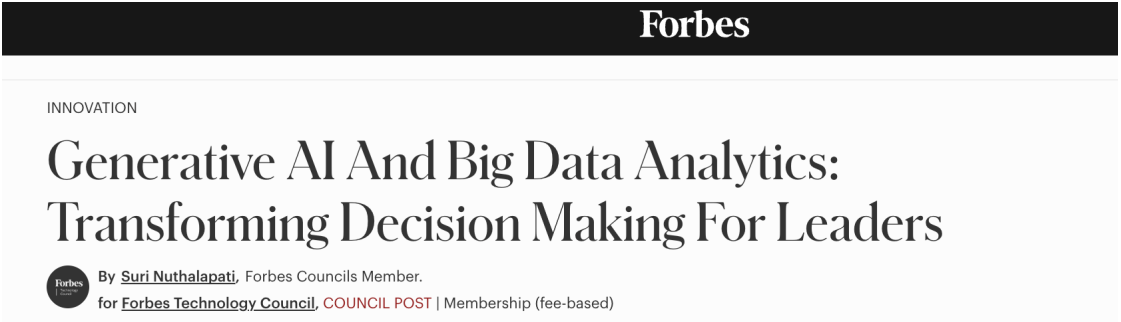
Problem



Value

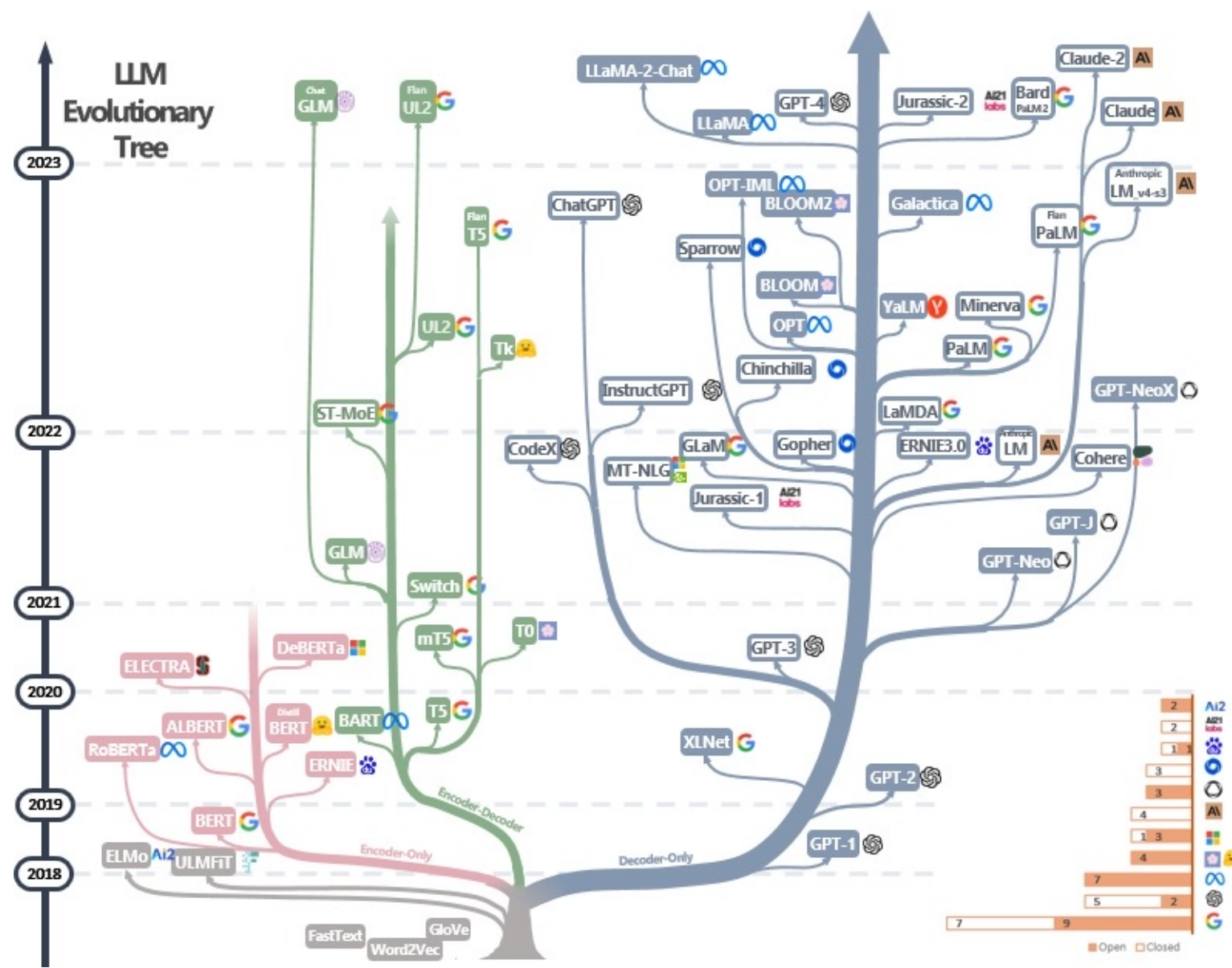
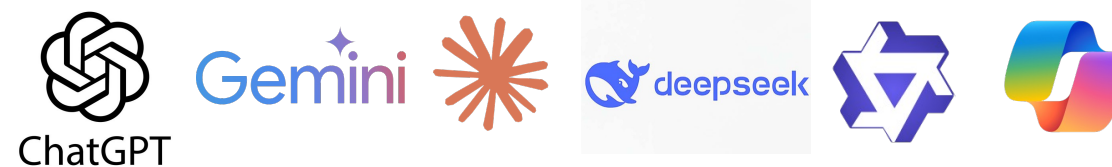


Future



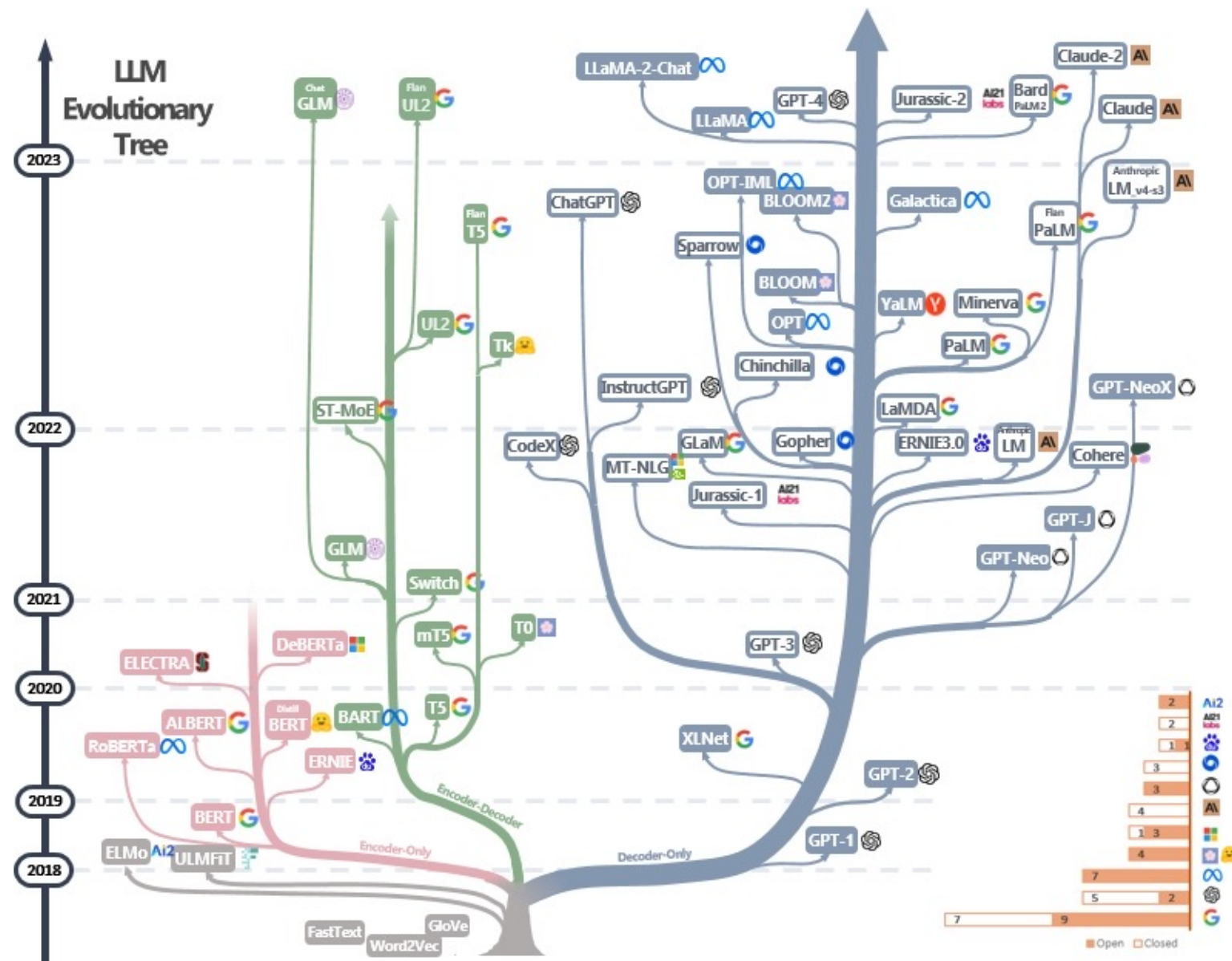
WHY TEXT MINING MATTERS?

- Large language models (LLMs) have made unstructured data analysis one of the most **popular** AI-applications and **available** to everyone
- LLMs have exploded



WHY TEXT MINING MATTERS?

— Where to put text mining in the LLM story?



Traditional text mining

- BoW
- TF-IDF
- SVD & LSI
- LDA



GOAL

- Transforming unstructured to structured data in the traditional way
 - Bag-of-Words (BoW)
 - TF-IDF
 - SVD and LSI
 - Word clouds
 - Word networks
 - Retweet analysis
- Unstructured data can also be photos or videos

Text representation

Textual analysis

GOAL

From text (here: tweets) ...

[1] "RT @SanjaysahFCCA: Covid-19: Nearly 100,000 catching virus every day.\n\n#Covid19UK #COVID19 #England #virus #pandemic #bbc news #Latest @BBCN"

[2] "Rang to get a blood test at the drs (iron levels low after giving birth 3 months ago) asked if I really need it as <https://t.co/7tTZms0jT9>"

[3] "RT @AaronLeeBee: 12 billion! 12 billion for track and trace. A test with loads of false positives and a bunch of kids on the phones. what"

[4] "12 billion! 12 billion for track and trace. A test with loads of false positives and a bunch of kids on the phones <https://t.co/QwivZZpxBb>"

[5] "RT @JohannaSaunders: Ive told family/friends that Im not doing Christmas/Presents this year, the best gift I can think of is nobody dying"

[6] "In support of businesses and individuals, the UK governments topped charging VAT on PPE during the beginning of the <https://t.co/JrpcyenuJ9>"

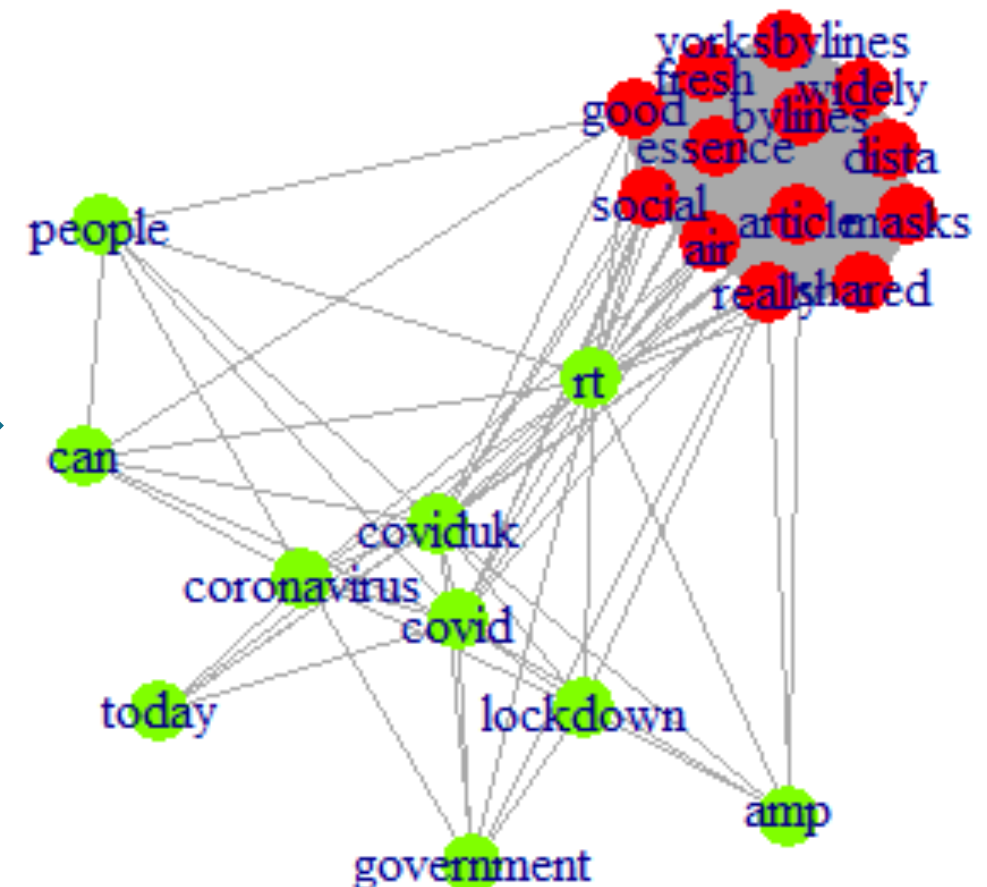
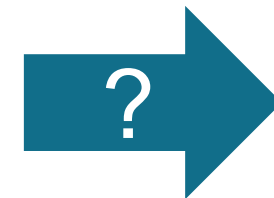
[7] "RT @jlbwhitesnake: @BBCScotland @BBCGarryR how can this be possible ? #BBCbias #Covid19UK #COVID19 #COVID__19 #coronavirus <https://t.co/w>"

[8] "RT @JamieKay22: Is it time for a national lockdown? #Covid19UK #COVID19 #coronavirus"

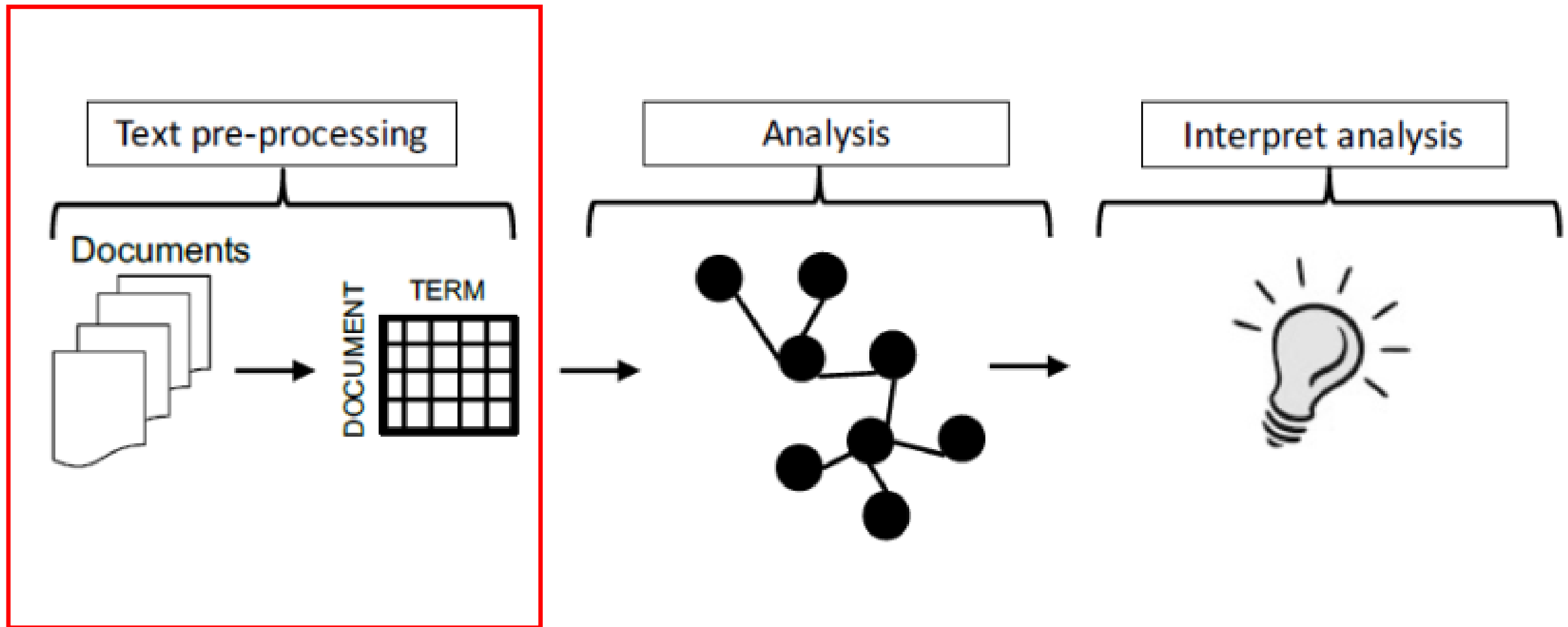
[9] "RT @BorisJohnson_MP: It is important to learn from the mistakes we made during the first wave of Coronavirus. So instead of delaying lockdo"

[10] "RT @jlbwhitesnake: @ScotTories @Conservatives #Covid19UK #COVID19 #coronavirus @SkyNews @BBCNews @talkRADIO @LBC @RuthDavidsonM"

To insights



OVERVIEW



TEXT MINING: BOW AND TF-IDF

FROM UNSTRUCTURED TO STRUCTURED

- **Goal:** take a collection of documents – each of which is a relatively free-form sequence of words – and turn it into a feature-vector representation
- A collection of documents = **corpus**
- A document is composed of individual **tokens** or terms
- Each document is one instance
 - Which features will be used is still to be determined

BAG-OF-WORDS

- KISS – Keep It Simple, Stupid
- Treat every document as just a collection of individual words
 - Ignore grammar, word order, sentence structure, and (usually) punctuation
 - Treat every word in a document as a potentially important keyword of the document
- What will be the feature's value in a given document?
 - Each document is represented by a one (if the token is present in the document) or a zero (the token is not present in the document)
- Inexpensive to generate & tends to work well for many tasks

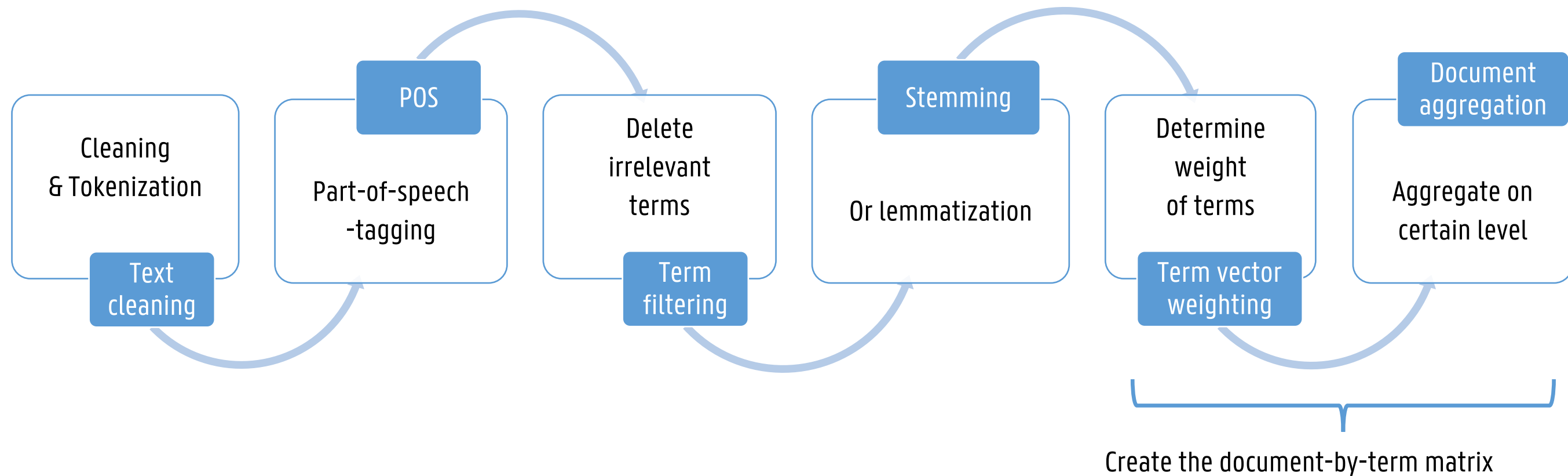
BAG-OF-WORDS

ID	Text
1	I am learning a lot about big data
2	Learning big things about big data
3	Studying big data is fun



ID	I	Am	Learning	A	Lot	About	Big	Data	Things	Studying	Is	Fun
1	1	1	1	1	1	1	1	1	0	0	0	0
2	0	0	1	0	0	0	1	1	1	0	0	0
3	0	0	0	0	0	0	1	1	0	1	1	1

TEXT PREPROCESSING PROCESS



TEXT CLEANING AND TOKENIZATION

— Normalization

- The case should be normalized so that every term is in lowercase
- Junk (punctuation, symbols, emoticons, hashtags, ...) can be removed

— Other cleaning step

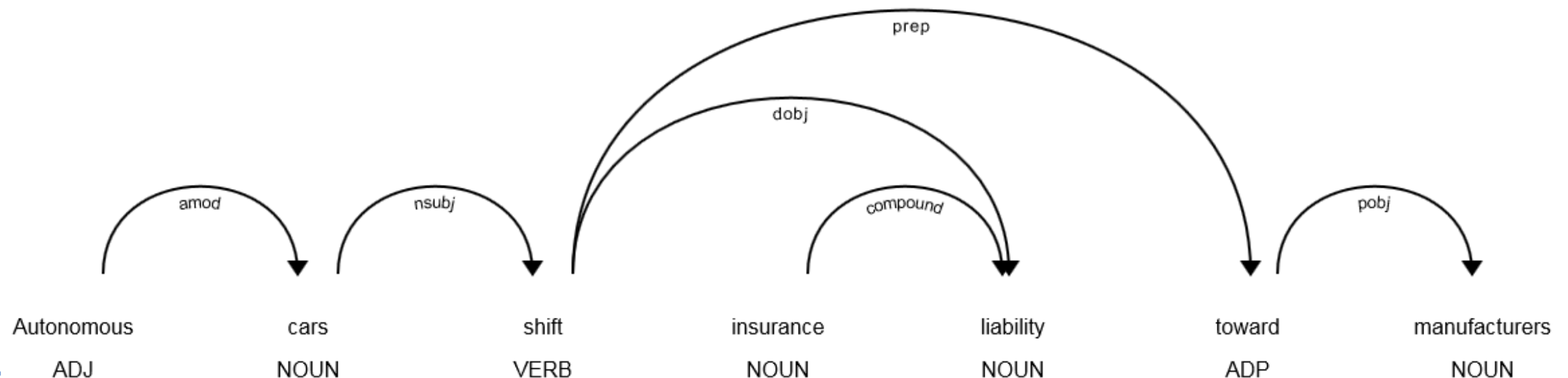
- Number removal or conversion
- Spell checking (especially important in sentiment analysis)

— Tokenization

- Converting the stream of characters to a stream of individual words or tokens
- White space is used to detect individual words and redundant white space should be removed

PART-OF-SPEECH TAGGING

- The process of labeling each word in a document with its syntactic category (called tags)
 - E.g., adjective, noun, verb, adverb, etc.
- Tags are dependent upon the use POS dataset and tagger
 - Popular datasets: Penn Treebank or OntoNotes 5.0
 - Modern approach: pre-trained neural network models
- Informative vs non-informative tags
- Often used in downstream tasks
 - Feature engineering or reduction in machine learning
 - Named entity recognition
 - Dependency analysis



TERM FILTERING

- Term filtering can be seen as a form of dimensionality reduction, otherwise even a small corpus would easily contain thousand words
- Term frequencies often follow *Zipf's law*
 - The term frequency is proportional to $\frac{1}{rank^p}$ where *rank* is the rank of the term in the sorted frequencies and *p* a fitting index close to 1
- Remove stop words
 - A stop word is a very common word in English (or whatever language is being parsed)
 - Words that appear frequently in the corpus and don't have discriminatory power
 - Typical words such as the words *the*, *and*, *of*, and *on* are removed
 - Can be dependent on the context!
- Remove rare words
 - Words that appear so infrequently that they do not help in future document description
 - Several options: delete words that only appear 1 or 2 times, only keep top-10% most frequent terms, only use informative POS tags, manual checking, etc.

STEMMING

- Stemming
 - Affixes (pre- and suffixes) are removed using a pre-defined rule (only root is kept)
 - E.g., noun plurals are transformed to singular forms
 - Snowball stemmers: Lovin and Porters stemming algorithm
 - Heavily depend on the pre-defined transformation rules
- More complex: lemmatization (e.g. better → good, flies/flight → fly)
 - Context is required: part of speech (PoS) tagging and dependencies
 - Words are brought back to their morphological root form or canonical form

STEMMING

— Stemming

- Fast and easily implemented
- Good if accuracy is of minor importance
- Example:

— *Students like to study analytics and big data → Student like to studi analyt and big data*

— Lemmatization

- Stemming only removes inflected word forms
- Lemmatization replaces these inflected forms with their base root form
- Additional support of lexicon is needed
- Slower, but more accurate
- Example:
 - *Students like to study analytics and big data → Student like to study analytic and big datum*
 - *Other lexicon: Students like to study analytics and big data → Student like to study analytic and big data*

Stemming

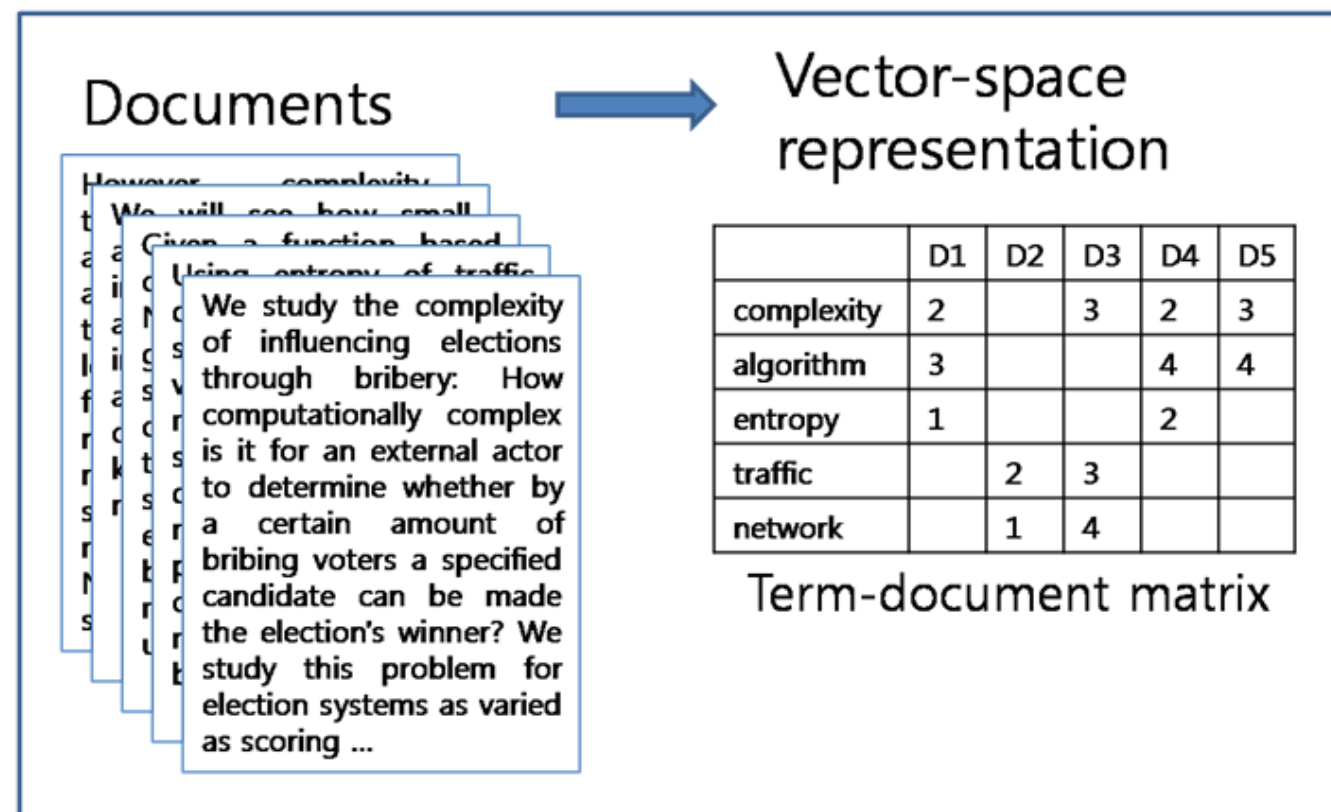
adjustable → adjust
formality → formaliti
formaliti → formal
airliner → airlin ⚠

Lemmatization

was → (to) be
better → good
meeting → meeting

DOCUMENT-BY-TERM MATRIX

- Document-by-term matrix (or term-by-document)
 - Use the term frequency (word count) in the document instead of just a zero or one
 - Differentiates between how many times a word is used
- Documents of various lengths
 - Number of terms >>> number of documents



DOCUMENT-BY-TERM MATRIX

ID	Text
1	I am learning a lot about big data
2	Learning big things about big data
3	Studying big data is fun

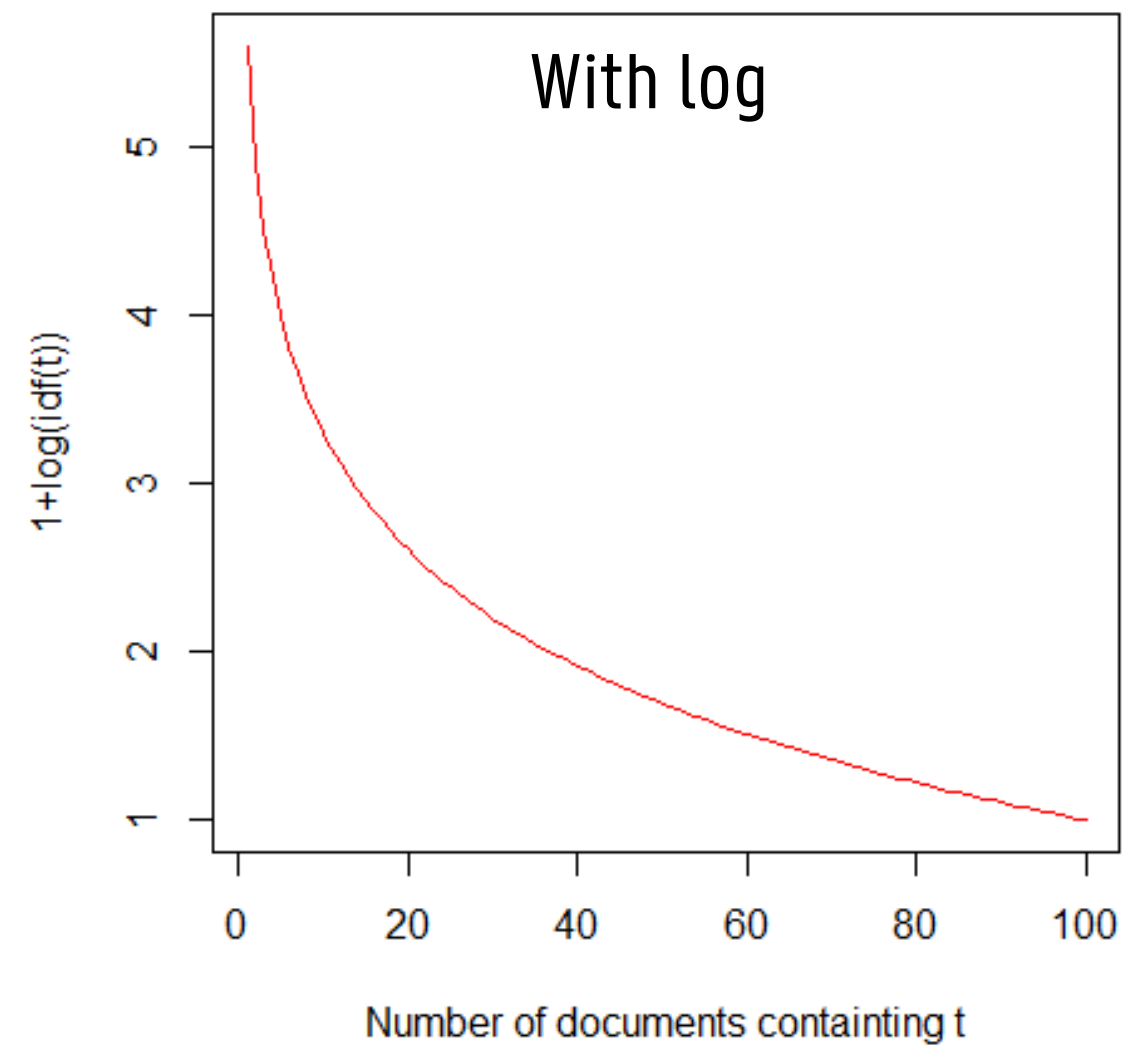
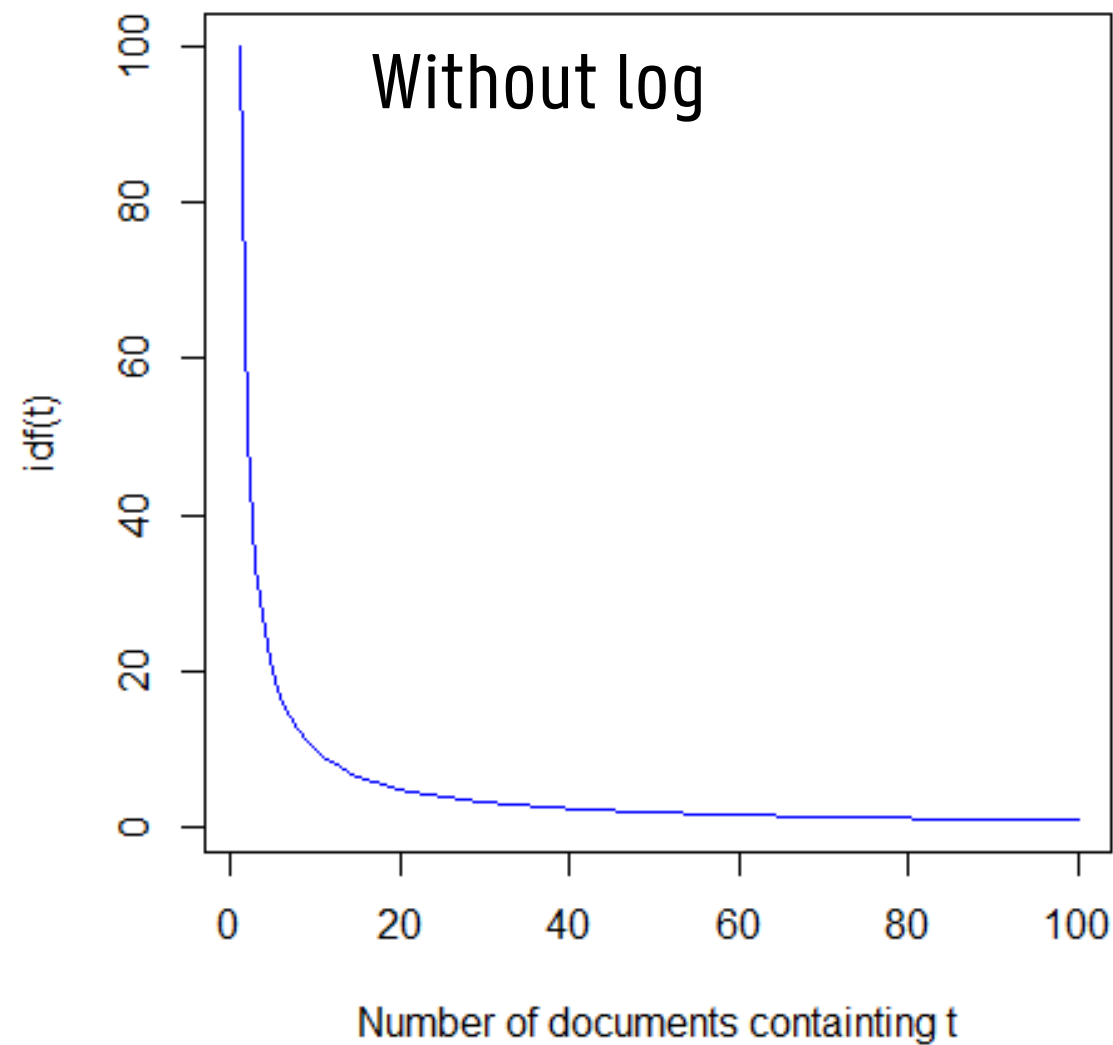


ID	I	Am	Learning	A	Lot	About	Big	Data	Things	Studying	Is	Fun
1	1	1	1	1	1	1	1	1	0	0	0	0
2	0	0	1	0	0	0	2	1	1	0	0	0
3	0	0	0	0	0	0	1	1	0	1	1	1

TERM WEIGHTING

- When deciding on the importance of terms, frequency in the corpus also plays a role
- Terms should not be too **rare**
 - Rare terms are only useful for information retrieval, not for classification or clustering
- Terms should not be too **common**
 - Common terms are not useful for classification, clustering
- Sparseness of a term measured by its Inverse Document Frequency
 - $IDF(t) = \frac{\text{Total number of documents}}{\text{Number of documents containing } t}$
 - $IDF(t) = 1 + \log\left(\frac{\text{Total number of documents}}{\text{Number of documents containing } t}\right)$

TERM WEIGHTING



TERM WEIGHTING

- Various weighting schemes are also available for the term frequency
 - $TF(t, d)$ represent the term frequency of term t in document d
- Binary term frequency weighting
 - $$\begin{cases} 0 & \text{if } TF(t, d) = 0 \\ 1 & \text{if } TF(t, d) > 0 \end{cases}$$
- Augmented normalized weighting
 - $$0.5 + 0.5 \frac{TF(t, d)}{\max(TF(t, d))}$$
- Logarithmic weighting
 - $\log(1 + TF(t, d))$

TF-IDF

- To get the final document-by-term matrix the term frequency and inverse document frequency are combined into the **TF-IDF**
- Raw TF-IDF
 - $w(t, d) = TF(t, d) \cdot IDF(t)$
- Logarithmic TF-IDF
 - $w(t, d) = \log(1 + TF(t, d)) \cdot (1 + \log(IDF(t)))$
- Gives you a weight for each term in each document
 - Reward terms that occur frequently in the document
 - Penalize for terms occurring frequently in the collection

TF-IDF

— Document weighting

	Term 1	Term 2	Term 3	Term 4	Term 5
Doc 1	0	2	2	6	8
Doc 2	1	2	0	0	7
Doc 3	3	2	0	6	8
Doc 4	0	0	0	0	7

— $TF\text{-}IDF = TF * IDF$

- Terms frequency * inverse document frequency
- $IDF = \frac{1}{\log(\text{total number of documents containing term } i)}$

	Term 1	Term 2	Term 3	Term 4	Term 5
Doc 1	0	2,67	8	12	8
Doc 2	2	2,67	0	0	7
Doc 3	6	2,67	0	12	8
Doc 4	0	0	0	0	7

$$8 * (4/4)$$

TF-IDF

— $\text{Log}(1+\text{TF})$

	Term 1	Term 2	Term 3	Term 4	Term 5
Doc 1	0	1.10	1.10	1.95	2.20
Doc 2	0.69	1.10	0	0	2.08
Doc 3	1.39	1.10	0	1.95	2.20
Doc 4	0	0	0	0	2.08

— $\text{Log}(1+\text{TF}) * (1 + \text{Log}(\text{IDF}))$

	Term 1	Term 2	Term 3	Term 4	Term 5
Doc 1	0	1.42	2.63	3.30	2.20
Doc 2	1.17	1.42	0	0	2.08
Doc 3	2.35	1.42	0	3.30	2.20
Doc 4	0	0	0	0	2.08

DOCUMENT AGGREGATION

- Depending on the application, the textual information often has to be aggregated to the desired level of analysis
 - For example, the individual complaints must be aggregated on customer level in order to be linked
- The aggregated document-by-term matrix:
 - $Aw(t, j) = \sum_{d=1}^r w(t, d)$
 - With $Aw(t, j)$ the aggregated weight of term t belonging to customer j , and r the number of documents belonging to customer j

PROBLEMS WITH TF-IDF

- Sparse matrix
 - Sparsity level: the number of empty cells (i.e., zeros) in a matrix
 - In real-life application sparsity levels are often $> 90\%$!
- Number of predictors $>$ number of observations (“curse of dimensionality”)
- Data reduction is necessary to avoid overfitting (i.e., model is learning data points too well)

TEXT MINING STEPS

- To recapitulate, the essential steps in text mining are:
 1. Text preprocessing
 2. Create document-by-term matrix
 3. Term weighting
 4. Document weighting
 5. Reduce the size of the dtm
 - PCA
 - Singular value decomposition
 - Non-negative matrix factorization
 - ...

TEXT MINING: SVD

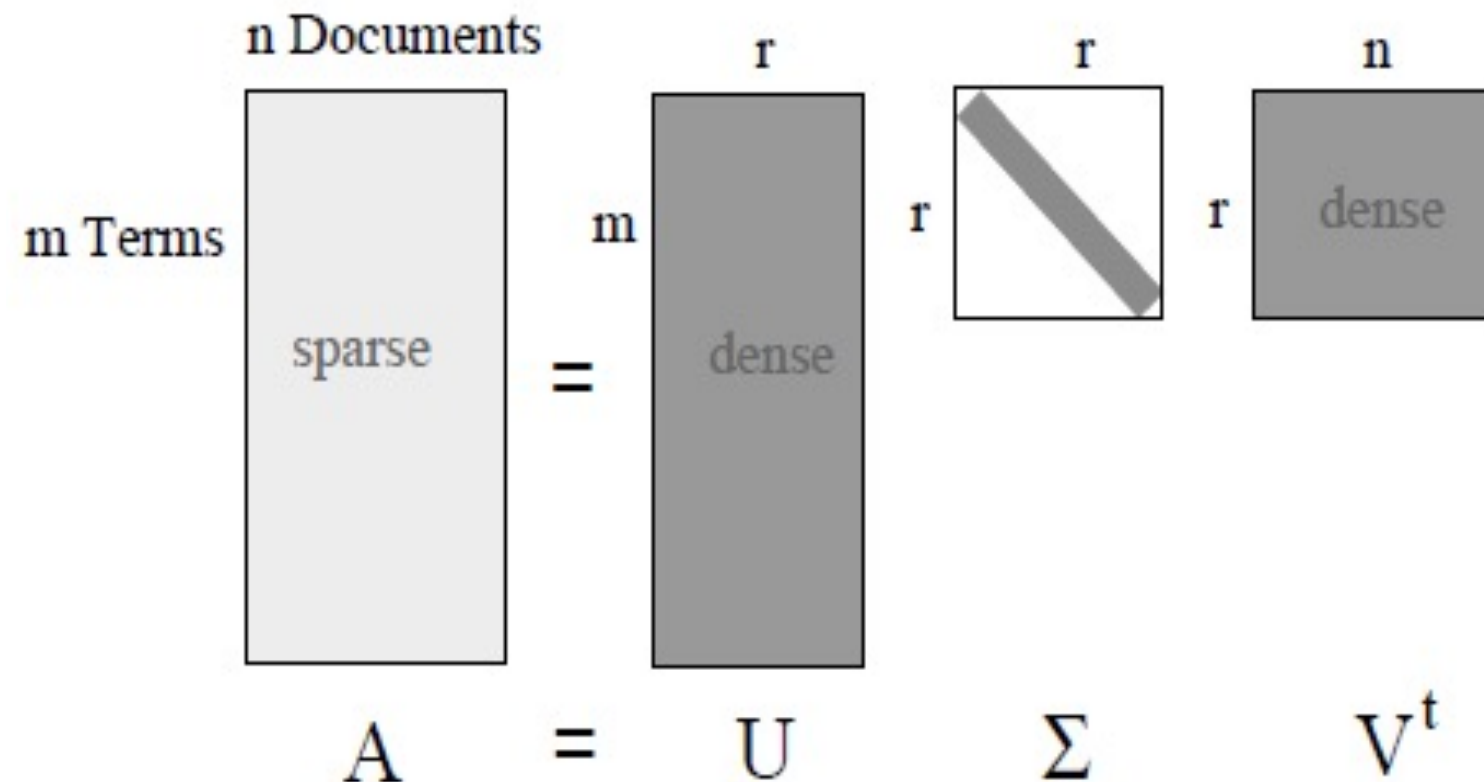
DIMENSIONALITY REDUCTION

- Dimensionality reduction
 - Reduce size of the dtm using latent semantic indexing (LSI)
 - Idea behind LSI: words that are used in the same context have similar meanings
 - LSI uses singular value decomposition
- **Singular value decomposition (SVD)**
 - Decompose matrix A into a unique product of 3 matrices
 - Reveal the latent factors in A by optimizing a performance metric (e.g., RMSE)

DIMENSIONALITY REDUCTION

- Matrix refresher
 - Rank of a matrix
 - The max number of linearly independent rows/columns
 - The number of non-zero-rows/columns in the echelon form
 - Vectors are orthogonal if the dot product is 0
 - $[0, 4, 6] \cdot [2, -3, 2]^t = 0 \times 2 + 4 \times (-3) + 6 \times 2 = 0$
 - Unit vector has a norm of 1
 - Norm: square root of the sum of squares
 - $\left[\frac{1}{\sqrt{2}} \quad \frac{1}{\sqrt{2}} \right] = \frac{1}{2} + \frac{1}{2} = 1$
 - Orthonormal basis: set of unit vectors that are pairwise orthogonal

SINGULAR VALUE DECOMPOSITION (SVD)



- U : terms-by-concept matrix, $d(\Sigma)$ weight of the concepts, V : concept-by-documents matrix
- r is the rank matrix A , U and V are column orthonormal, V^T has orthonormal rows, Σ is diagonal matrix with singular values

SVD

- SVD represents A as the product of 3 specific matrices.
- How can these matrices be interpreted?
 - U represents how much a given term corresponds to a given concept or dimension
 - V represents how much a given document corresponds to a given concept or dimension
 - Σ a square matrix containing the singular values on the diagonal. Singular values are positive and sorted in decreasing order of the amount of variance they account for in the original data. Hence, they represent the strength of each concept.

SVD: EXAMPLE

- Assume that we want to calculate the eigenvalues of the 4x2 matrix A

$$- A = \begin{bmatrix} 2 & 4 \\ 1 & 3 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

- For this matrix A, we know that a NxN matrix W, we can find a nonzero vector $\mathbf{x}$ for which holds:
 - $W\mathbf{x} = \lambda\mathbf{x}$ in which λ are the eigenvalues and $\mathbf{x}$ the eigenvector of A corresponding to λ
- The goal is now to find the eigenvalues of AA^T and $A^T A$
- The eigenvectors of AA^T make up the U matrix, whereas the eigenvectors of $A^T A$ make up the V matrix

SVD: EXAMPLE

— Let's find U

$$- AA^T = \begin{bmatrix} 2 & 4 \\ 1 & 3 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 2 & 1 & 0 & 0 \\ 4 & 3 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 20 & 14 & 0 & 0 \\ 14 & 10 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = W$$

— Now that we have a square matrix, we can use the formula $W\mathbf{x} = \lambda\mathbf{x}$ or $(W - \lambda I)\mathbf{x} = 0$

$$- \begin{bmatrix} 20 - \lambda & 14 & 0 & 0 \\ 14 & 10 - \lambda & 0 & 0 \\ 0 & 0 & 0 - \lambda & 0 \\ 0 & 0 & 0 & 0 - \lambda \end{bmatrix} \mathbf{x} = 0$$

— If you now solve the characteristic equation $|W - \lambda I| = 0$, you get 4 eigenvalues $\lambda = 0, \lambda = 0, \lambda = 29.866, \lambda = 0.134$

— Next, you should solve the above equation using the different eigenvalues, to get all the columns of the U matrix

SVD: EXAMPLE

— Let's find U

- For example to find the first column of the U-matrix, substitute $\lambda = 29.866$ in the equation and solve for all values of x
 - $-9.866x_1 + 14x_2 = 0$
 - $14x_1 - 19.866x_2 = 0$
 - $x_3 = 0$
 - $x_4 = 0$
- The solution then becomes: $x_1 = 0.817, x_2 = 0.576, x_3 = x_4 = 0$
- The second column can be found by substituting $\lambda = 0.134$

— The final U matrix becomes:
$$U = \begin{bmatrix} 0.82 & -0.58 & 0 & 0 \\ 0.58 & 0.82 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

SVD: EXAMPLE

- To find the V matrix, do a similar analysis

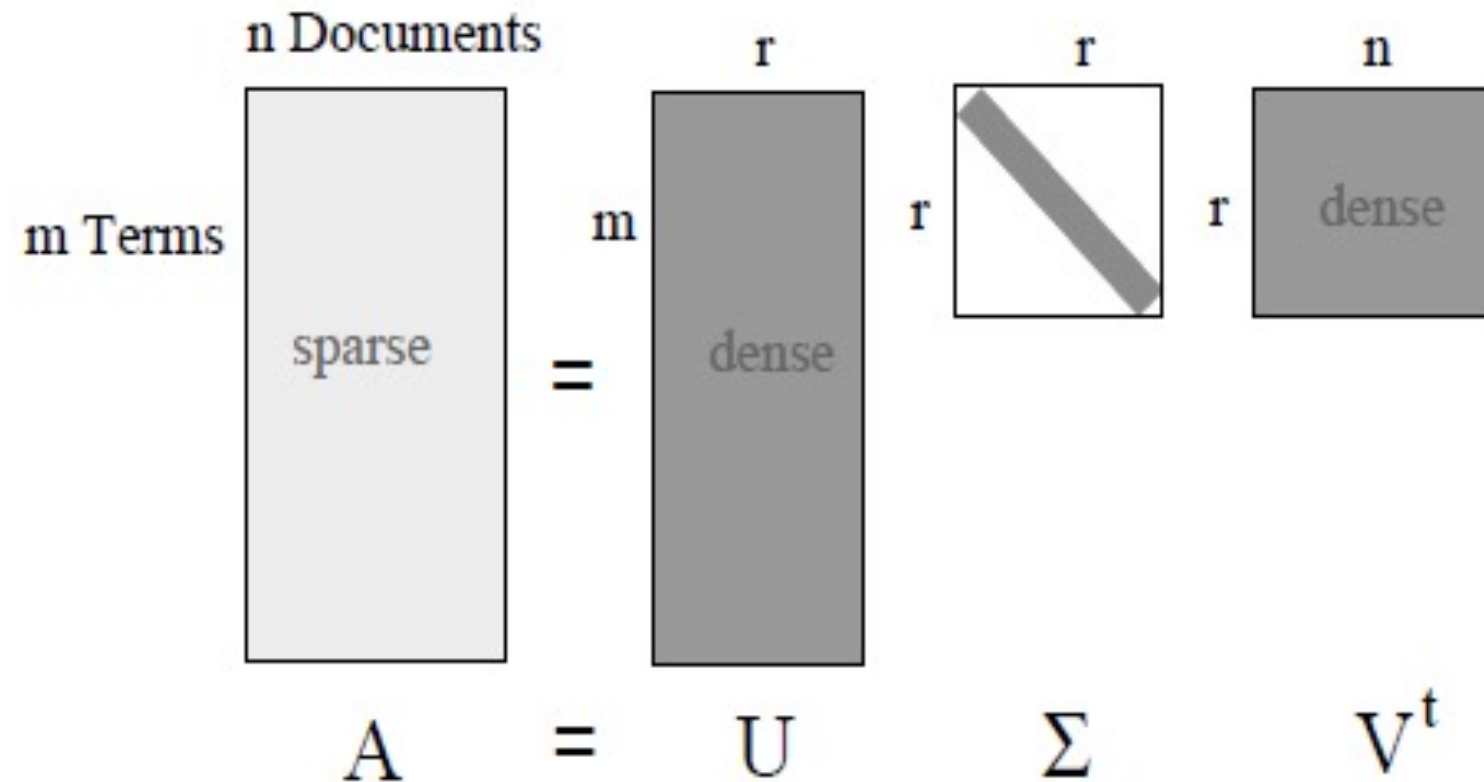
$$- A^T A = \begin{bmatrix} 2 & 1 & 0 & 0 \\ 4 & 3 & 0 & 0 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 1 & 3 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$- \text{And you obtain: } V = \begin{bmatrix} 0.40 & -0.91 \\ 0.91 & 0.40 \end{bmatrix}$$

- Remember to r-matrix you should take the square root of the eigenvalues (=singular values)

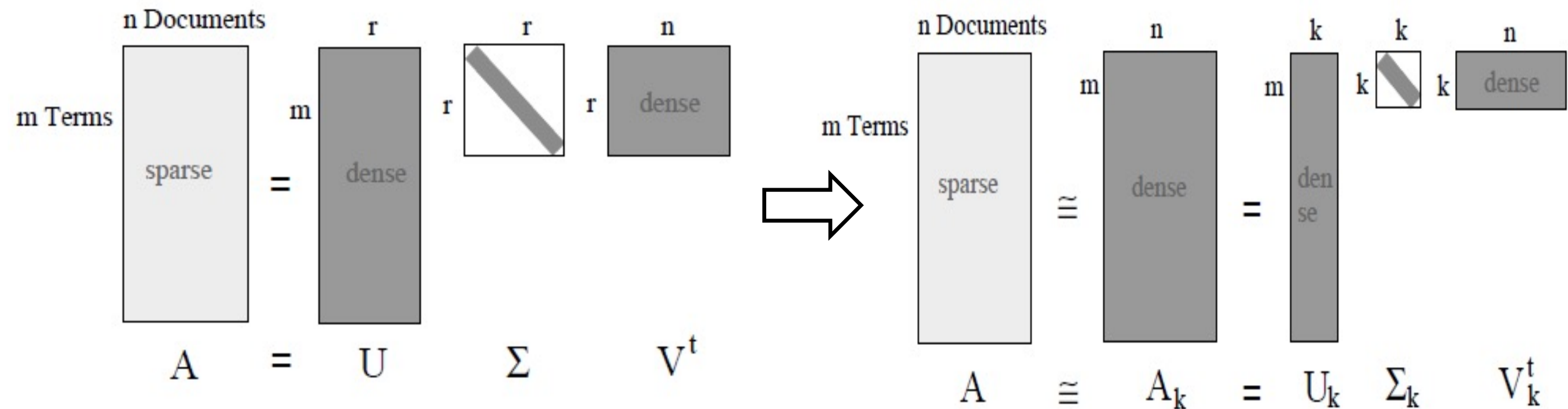
$$- r = \begin{bmatrix} 5.47 & 0 \\ 0 & 0.37 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

FROM SVD TO LSI



- Σ is $r \times r$ diagonal matrix with singular values
- Want to make r smaller by setting smallest singular values to 0
 - renders corresponding columns of U and V useless and gives more compact representation

LSI



- LSI implies the optimal number of latent concepts k (i.e., dimensionality is reduced as $k < r$)
 - Σ_k retains first k singular values (i.e., k most important concepts)
 - U and V are truncated. This means that the useless concepts and columns from U and rows from V are deleted
 - Multiplication of truncated matrices $\approx A$

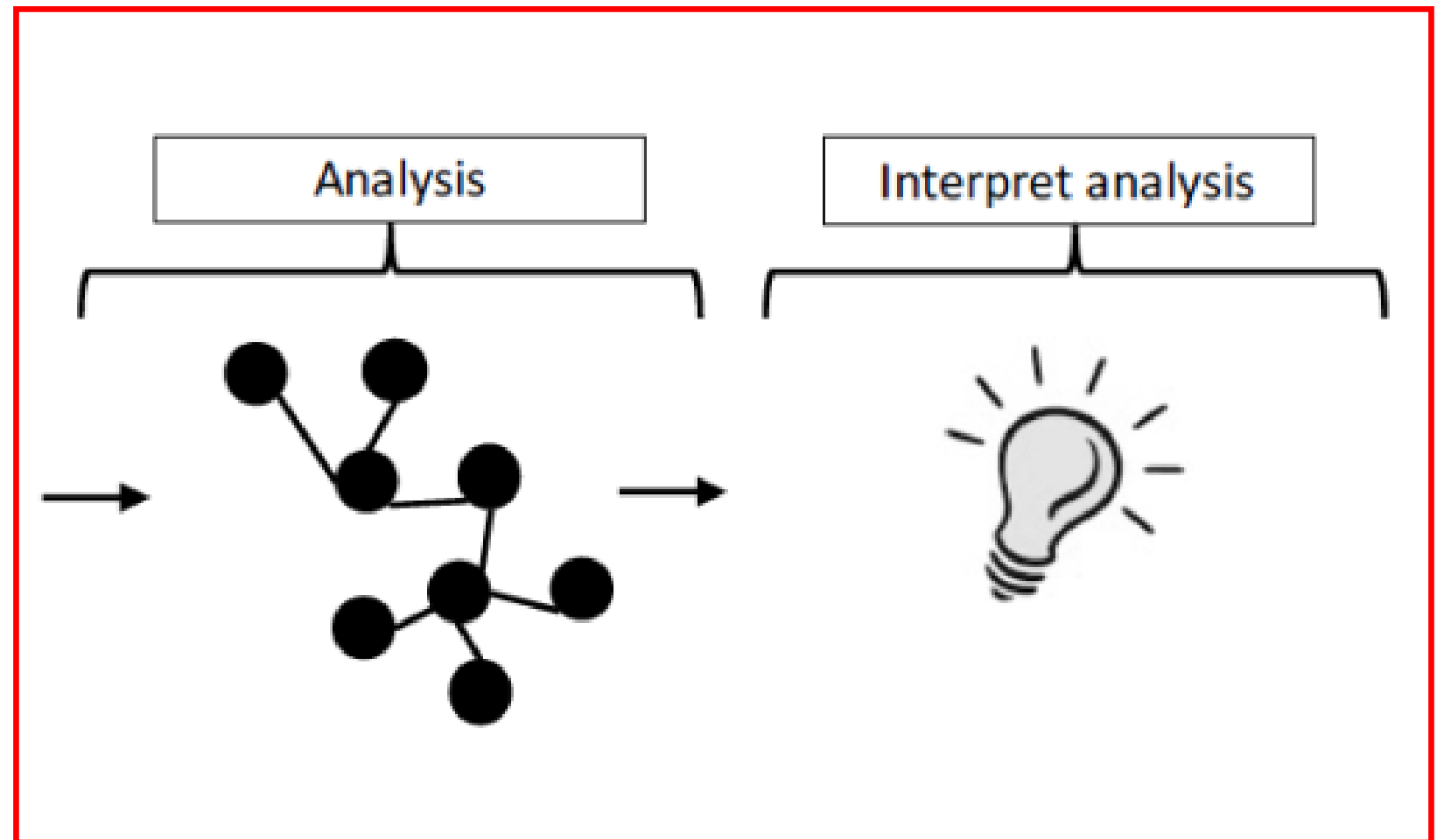
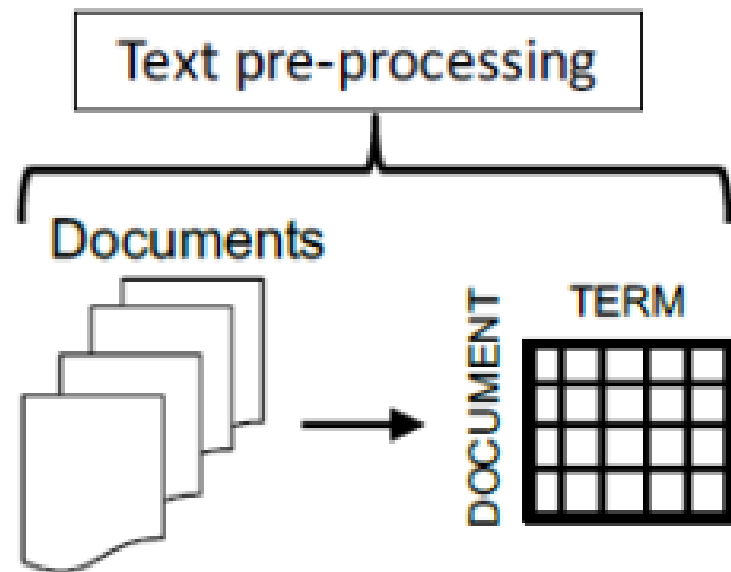
- How to select the optimal number of dimensions (k)?
 - Percentage energy explained
 - Energy = sum of squares of singular values
 - Choose k such that 90% of the energy is retained
 - Percentage explained variance
 - Scree plot method: the eigenvalues are plotted in descending order and one must find the “elbow” in the graph
 - Cross-validation
- When selecting the optimal number of concept (k), new test instances can be calculated by:
 - $V_d = A'_d U_k \Sigma_k^{-1}$
 - U_k : the k -rank term-by-concept matrix, Σ_k^{-1} : the diagonal singular value matrix in rank k

TEXT MINING IN PYTHON

- **Data manipulation (Pandas & re)**
 - Text is stored in **DataFrames** to easily manipulate data alongside metadata
 - **Regular Expressions (re)** are used for advanced pattern matching, data cleaning, and extraction
 - Pandas easily integrates regex (via `.str`) for efficient, vectorized text filtering
- **Linguistic processing (NLTK & spaCy)**
 - **NLTK**: The standard library for tokenization, stop-word removal, and stemming
 - **spaCy**: Industrial-strength NLP for high-speed advanced tasks (like POS tagging, tokenization and Named Entity Recognition)
- **Feature extraction & modeling (Scikit-Learn)**
 - Converts text into numerical **Document-by-Term matrices** (=vectorization)
 - Standard pipelining interface for clustering, classification, and topic modeling algorithms

WORD ANALYSIS

OVERVIEW

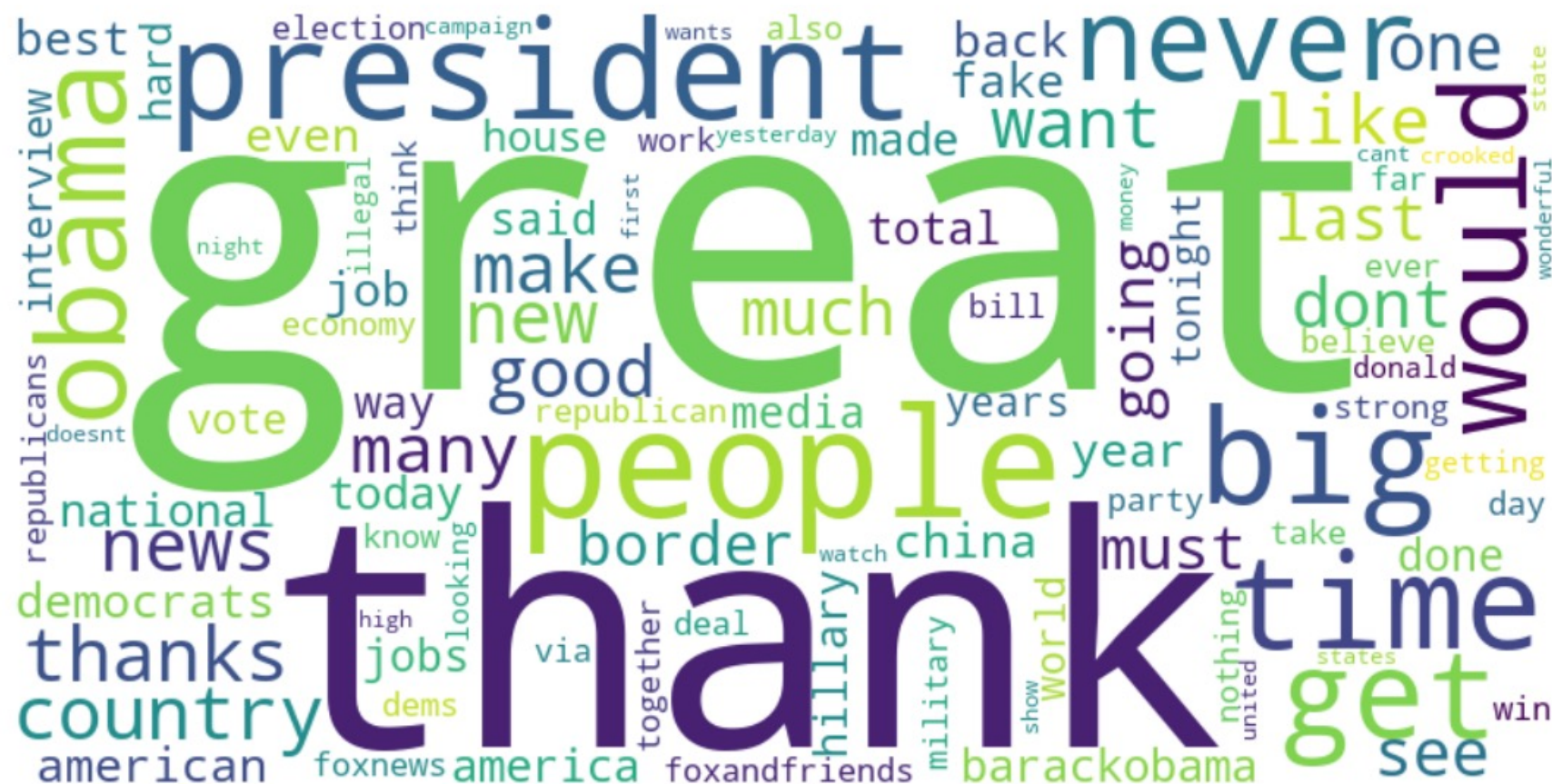


WORD ANALYSIS

— Word clouds

- Graphical depiction of the most frequent words
- The size of the word's text is in proportion with its frequency
- However, it is a Bag-of-Words approach: you see what is said, but not the context or which words belong together

Trump Tweets Word Cloud



WORD ANALYSIS

— Re-tweet analysis

- Helps companies understand what they should state in their posts
 - Which words work and which ones not? In other words, which words drive engagements?
 - **Dependent** variable: number of re-tweets (proxy for tweet popularity)
 - **Independent** variable: words in the tweets
 - By analyzing the word importance in a predictive model, we can identify the high-value words
 - For example, does the word 'free' increase re-tweeting?
- See [Lecture2_Notebook.ipynb](#) for an overview of word analysis in Python

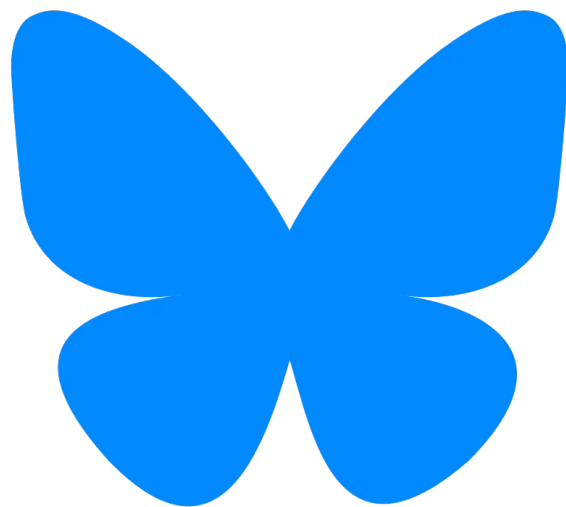


REMEMBER

Post about this lecture:

[#SMWA2026](#)

[#Lecture3](#)



SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert